A Mini Project Report Of

PCC-201-COM : Data Structures

Submitted by

MANOJ SUINDARLAL CHAVAN (53)

Under the Guidance of
Prof. Tejal H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Manoj Sundarlal Chavan (53),** on **"Implementation of Palindrome checker [Stacks and Queues]"** is work done by her and submitted during 2025–2026 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal H. Patil
(Guide)

Dr. Lalit V. Patil
Head Of Computer Engg

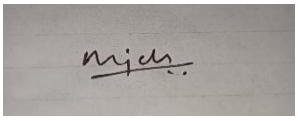
ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit V. Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

MANOJ SUNDARLAL CHAVAN (53)



ABSTRACT

A palindrome is a word, number, or sequence that reads the same forward and backward. Examples of palindromes include MADAM, LEVEL, and 12321. Palindrome checking is a simple application that demonstrates the use of fundamental data structures.

This project, **Implementation of Palindrome Checker Using Stacks and Queues**, aims to determine whether a given input is a palindrome by using stack and queue data structures. A stack follows the Last In, First Out (LIFO) principle, while a queue follows the First In, First Out (FIFO) principle.

In this project, each character of the input is inserted into both a stack and a queue. Characters are then removed from both data structures and compared. If all corresponding characters match, the input is identified as a palindrome. Otherwise, it is classified as not a palindrome.

The project is implemented in Python and demonstrates the practical application of stack operations, queue operations, and character comparison. It helps develop an understanding of linear data structures and their use in solving basic computational problems.

.

SR. NO.	CONTENT	PAGE NO.
1	Introduction	1
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design a) Existing system b) Proposed system c) ER Diagram d) System code	5
6	Result	9
7	Application	10
8	Future Scope	11
9	Conclusion	12
10	References	13

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	03
Fig 5.1	System Design	05
Fig 5.2	System Code	07
Fig 5.3	Result	09

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

1.1 Overview:

A palindrome is a word, number, or sequence that remains the same when read from left to right and from right to left. For example, MADAM, LEVEL, RADAR, and 12321 are palindromes, whereas HELLO and 12345 are not palindromes.

Stacks and queues are important linear data structures used in computer science. A stack follows the Last In, First Out (LIFO) principle, meaning that the last inserted element is removed first. A queue follows the First In, First Out (FIFO) principle, meaning that the first inserted element is removed first. In this project, stack and queue data structures are used together to check whether a given input is a palindrome. Each character is inserted into both data structures. The characters removed from the stack and queue are compared to determine whether the input reads the same in both directions.

1.2 Problem Statement :

Checking whether a word or number is a palindrome is a common programming problem. A simple method is to compare the input with its reverse. However, this project focuses on solving the problem using stack and queue data structures to understand their operations and differences.

Therefore, a Python-based palindrome checker is developed using stack and queue operations. The system compares characters removed from both data structures and displays whether the input is a palindrome.

1.3 Objective:

- To understand the functional properties and operations of Stacks (LIFO) and Queues (FIFO).
- To apply these data structures to solve a real-world string manipulation problem.
- To implement the palindrome-checking logic in Python.
- To demonstrate how the contrasting nature of stacks and queues can be used for string comparison.

LITERATURE SURVEY

Stacks and Queues are fundamental linear data structures extensively studied in computer science. A **Stack** operates on a **Last-In, First-Out (LIFO)** principle, making it ideal for tasks that require reversing order, such as in function call management (the "call stack") or undo-redo features. A **Queue** operates on a **First-In, First-Out (FIFO)** principle, which is suited for managing ordered tasks like print spooling, task scheduling, or breadth-first-search algorithms.

The problem of palindrome detection is a classic introductory problem for data structures. While it can be solved efficiently with a simple two-pointer string indexing approach, using a stack and queue provides a clear pedagogical demonstration of their opposing mechanisms. This method showcases how a stack inherently reverses the string's order, while a queue preserves it, allowing for a direct, sequential comparison of the string's first half with its reversed second half.

2. LITERATURE SURVEY

Sr. No.	Topic / Method	Description	Limitations / Advantages
1	Basic Palindrome Checking	A palindrome is a word or number that reads the same forward and backward. Examples include MADAM, LEVEL, and 12321.	Simple to understand and useful for learning string processing.
2	Palindrome Checking Using String Reversal	The input is reversed and compared with the original input. If both are equal, the input is a palindrome.	Easy to implement, but does not directly demonstrate stack and queue operations.
3	Stack-Based Palindrome Checking	A stack follows the Last In, First Out (LIFO) principle. Characters are pushed onto the stack and popped in reverse order.	Helps understand stack operations and reverse-order processing.
4	Queue-Based Palindrome Checking	A queue follows the First In, First Out (FIFO) principle. Characters are inserted and removed in their original order.	Helps understand queue operations and sequential processing.
5	Palindrome Checker Using Stack and Queue	The proposed system stores input characters in both a stack and a queue. It compares characters removed from the stack and queue to identify a palindrome.	Demonstrates both data structures and their practical use in solving a programming problem.

UML DIAGRAM

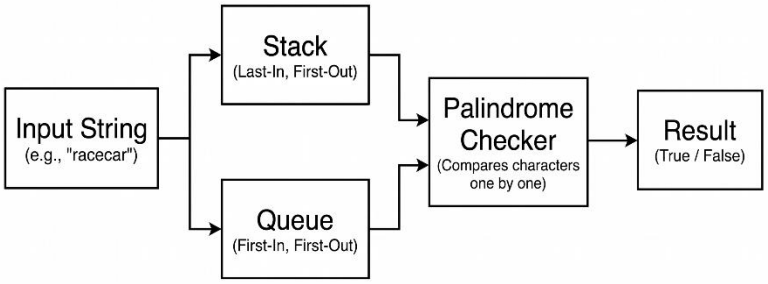


Fig 3.1: Use Case Diagram

SYSTEM REQUIREMENT

Hardware Requirements

- **Operating System:** Windows 10/11 (64-bit)
- **Processor:** Intel Core i3 or above
- **RAM:** 4 GB minimum
- **Hard Disk:** 256 GB or more

Software Requirements

- **Programming Language:** Python 3.x
- **IDE:** Jupyter Notebook / VS Code
- **Libraries:** collections (specifically deque for efficient stack and queue operations)

SYSTEM DESIGN

Existing System:

Traditional palindrome checking is often done using a **two-pointer approach**. One pointer starts at the beginning of the string (index 0) and another at the end (index n-1). The characters at both pointers are compared. If they match, the "start" pointer is incremented and the "end" pointer is decremented, moving them both toward the center. If a mismatch is found, the string is not a palindrome. This is very efficient but does not demonstrate the specific principles of LIFO/FIFO data structures.

Proposed System:

The proposed system uses a Stack and a Queue to validate the string, providing a clear illustration of their distinct functionalities.

1. **Initialization:** An empty stack and an empty queue are created. (In Python, collections.deque can be used for both).
2. **Processing:** The input string is iterated through, character by character. To make the check robust, we only process alphanumeric characters and convert them to lowercase.
3. **Filling:** Each processed character is **pushed** onto the stack (LIFO) and **enqueued** into the queue (FIFO).
4. **Comparison:** A loop runs as long as the stack (or queue) is not empty.
    - In each iteration, one character is **popped** from the stack. This will be the *last* character of the string first.
    - One character is **dequeued** from the queue. This will be the *first* character of the string first.
5. **Validation:** The character from the stack is compared to the character from the queue.
    - If they are **not equal** at any point, the loop is broken, and the string is declared "Not a Palindrome".
    - If the loop completes and all characters matched, the string is declared a "Palindrome".

Entity Relationship Diagram:

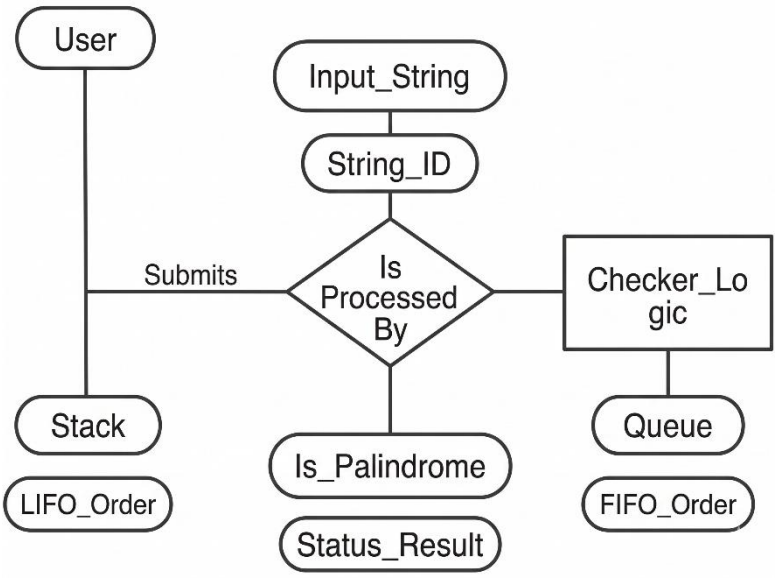


Fig 5.1: ER Diagram

SYSTEM CODE

```
class Stack:
    def __init__(self):
        self.elements = []

    def push(self, item):
        self.elements.append(item)

    def pop(self):
        if not self.is_empty():
            return self.elements.pop()
        return None

    def is_empty(self):
        return len(self.elements) == 0

    def size(self):
        return len(self.elements)


class Queue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item):
        self.elements.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.elements.pop(0)
        return None

    def is_empty(self):
        return len(self.elements) == 0

    def size(self):
        return len(self.elements)


class PalindromeChecker:
    def __init__(self):
        self.stack = Stack()
        self.queue = Queue()
```

```
    def check_palindrome(self, text: str) -> bool:
        cleaned_text = "".join(char.lower() for char in text if char.isalnum())

        if not cleaned_text:
            return False

        self.stack = Stack()
        self.queue = Queue()

        for char in cleaned_text:
            self.stack.push(char)
            self.queue.enqueue(char)

        while not self.stack.is_empty() and not self.queue.is_empty():
            from_stack = self.stack.pop()
            from_queue = self.queue.dequeue()

            if from_stack != from_queue:
                return False

        return True


def main():
    checker = PalindromeChecker()

    print("=" * 45)
    print("   Palindrome Checker (Stacks & Queues)   ")
    print("=" * 45)

    while True:
        user_input = input("\nEnter a string to check (or type 'exit' to quit): ").strip()

        if user_input.lower() == 'exit':
            print("Exiting program. Goodbye!")
            break

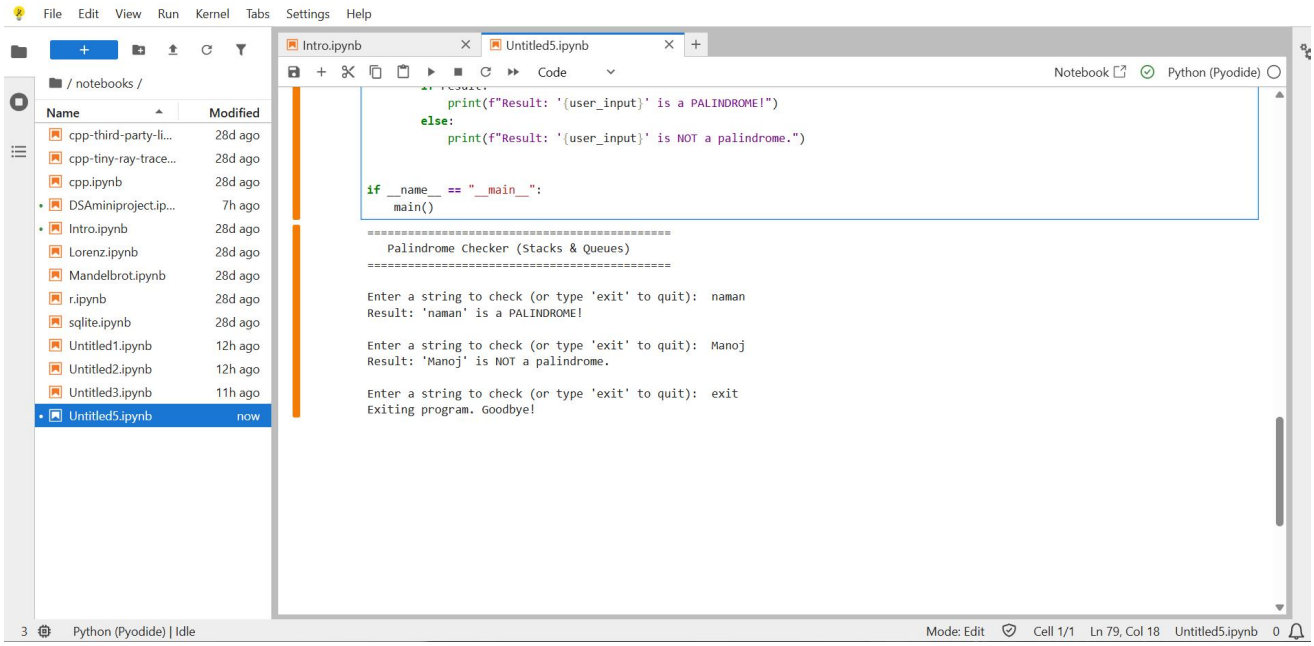
        if not user_input:
            print("Please enter a valid non-empty string.")
            continue

        result = checker.check_palindrome(user_input)
```

```
if result:
            print(f"Result: '{user_input}' is a PALINDROME!")
        else:
            print(f"Result: '{user_input}' is NOT a palindrome.")


if __name__ == "__main__":
    main()
```

RESULT


File Edit View Run Kernel Tabs Settings Help
/ notebooks /
Name
Modified
cpp-third-party-li... 28d ago
cpp-tiny-ray-trace... 28d ago
cpp.ipynb 28d ago
DSAminiproject.ip... 7h ago
Intro.ipynb 28d ago
Lorenz.ipynb 28d ago
Mandelbrot.ipynb 28d ago
r.ipynb 28d ago
sqlite.ipynb 28d ago
Untitled1.ipynb 12h ago
Untitled2.ipynb 12h ago
Untitled3.ipynb 11h ago
Untitled5.ipynb now
Intro.ipynb
Untitled5.ipynb
Code
Notebook
Python (Pyodide)
print(f"Result: '{user_input}' is a PALINDROME!")
else:
print(f"Result: '{user_input}' is NOT a palindrome.")
if __name__ == "__main__":
main()
Palindrome Checker (Stacks & Queues)
Enter a string to check (or type 'exit' to quit): naman
Result: 'naman' is a PALINDROME!
Enter a string to check (or type 'exit' to quit): Manoj
Result: 'Manoj' is NOT a palindrome.
Enter a string to check (or type 'exit' to quit): exit
Exiting program. Goodbye!
Python (Pyodide) | Idle
Mode: Edit
Cell 1/1
Ln 79, Col 18
Untitled5.ipynb

APPLICATIONS

This project, while simple, demonstrates a core concept with several applications:

- **Educational Tool:** It serves as an excellent academic resource for teaching the fundamental differences between LIFO (Stack) and FIFO (Queue) data structures in a practical, easy-to-understand context.
- **Text Processing:** The underlying principle can be used in more complex text analysis and data validation tools where reversing or comparing sequences is necessary.
- **Algorithmic Basis:** The concept of using a stack to reverse a sequence is a foundational technique used in many other algorithms, such as expression evaluation (infix to postfix) or depth-first search.
- **Bioinformatics:** Palindromic sequences are important in genetics (e.g., DNA restriction sites). While more complex algorithms are used, this project models the basic concept of identifying such symmetric sequences.

FUTURE WORK

- **Graphical User Interface (GUI):** The project can be enhanced with a graphical interface using Python libraries like **Tkinter** or **Pygame**. This would allow a user to type in a string and see the result visually.
- **Real-time Visualization:** A future version could animate the process, showing the stack and queue being filled and then emptied, with a visual comparison of the characters as they are popped and dequeued.
- **Performance Analysis:** Implement the two-pointer approach and compare its execution time against this stack/queue method for very large strings to analyze the time and space complexity differences.
- **Extended Functionality:** The program could be modified to find the longest palindromic substring within a larger string, which is a more complex algorithmic challenge.

CONCLUSION

In this mini-project, a classic Palindrome Checker was successfully modeled and implemented using **Stack and Queue** data structures. The algorithm efficiently determines if an input string is a palindrome by leveraging the LIFO property of a stack and the FIFO property of a queue.

Through this implementation, fundamental data structure concepts were applied in a practical and interactive way. The project clearly demonstrates the opposing mechanisms of stacks and queues and how they can be combined to solve a common problem.

This mini-project not only strengthens the theoretical knowledge of data structures but also provides a foundation for understanding more complex algorithms that rely on these foundational concepts, such as parsers, schedulers, and search algorithms.

REFERENCE

- **GeeksforGeeks** - "Check if a string is palindrome using Stack" https://www.geeksforgeeks.org/check-if-a-string-is-palindrome-using-stack/
- **Python Software Foundation** - Python 3 Documentation (collections.deque) https://docs.python.org/3/library/collections.html#collections.deque
- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C.** - *Introduction to Algorithms (CLRS)* (For fundamental definitions of Stack and Queue data structures).
- **TutorialsPoint** - "Data Structure - Stacks" and "Data Structure - Queues" https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.html